

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of Mon 7 FEBRUARY 2022 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

SIGNATURE:

(or PERSON CODE)

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.00m

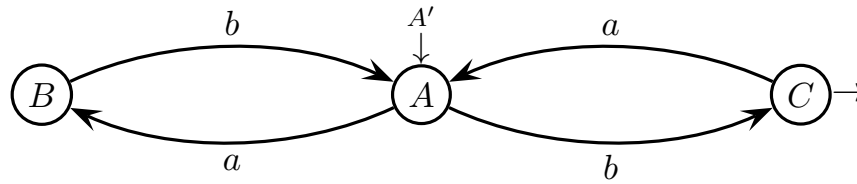
1 Regular Expressions and Finite Automata 20%

1. Consider the following regular expression R , over the two-letter alphabet $\Sigma = \{ a, b, \}$:

$$R = (a b \mid b^+ a)^* b$$

Answer the following questions:

- (a) Write the short strings of length ≤ 4 generated by the regular expression R .
- (b) By using the Berry-Sethi method, find a deterministic automaton A equivalent to the regular expression R .
- (c) Determine if automaton A is minimal, explain your answer, and if necessary minimize the automaton.
- (d) Determine if the language generated by the regular expression R is local or not, and explain your answer.
- (e) Write a strictly right-unilinear grammar G' equivalent to the automaton A' shown below:



- (f) (optional) By using the language equation method, find a regular expression R' equivalent to automaton A' , starting from the strictly unilinear grammar G' found at the previous point.
-

Solution

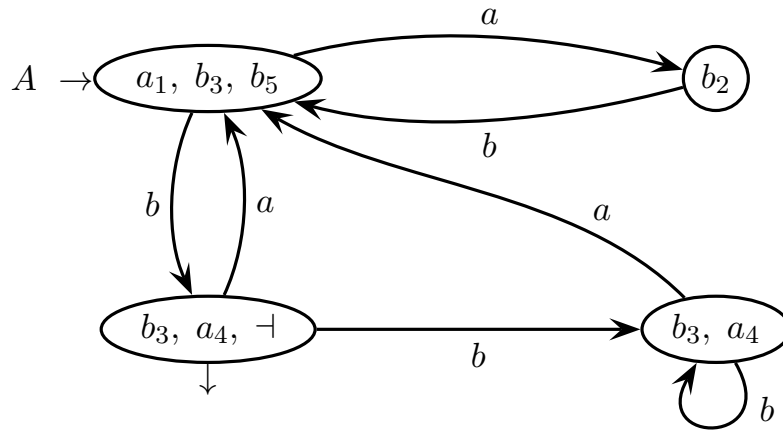
(a) Here are the four valid short strings of length less than or equal to 4:

$b \quad a b b \quad b a b \quad b b a b$

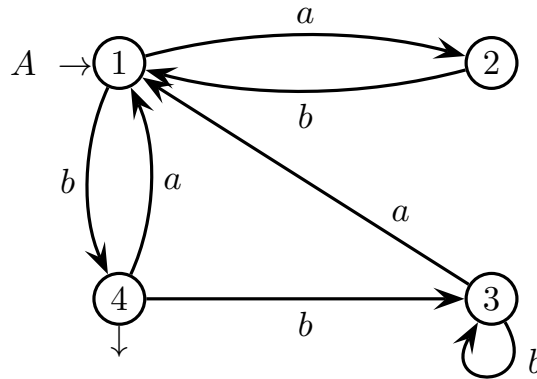
(b) Here is the Berry-Sethi (BS) method applied to the regular expression R :

$$R_{\#} = (a_1 b_2 \mid b_3^+ a_4)^* b_5 \dashv$$

initial(s)	a_1, b_3, b_5
generator	follower(s)
a_1	b_2
b_2	a_1, b_3, b_5
b_3	b_3, a_4
a_4	a_1, b_3, b_5
b_5	$\dashv$



After renaming the states, automaton A becomes:



Automaton A is clean and deterministic by construction. It may be non-minimal.

- (c) Automaton A is deterministic, so minimization theory can be applied. Automaton A is minimal, because all the states are distinguishable: state 4 is final while the others are not; state 2 does not have an outgoing a -arc while states 1 and 3 have it; by input b , state 1 goes to a final state while state 3 does not. Therefore all the states are pairwise distinguishable and the automaton is minimal.
- (d) Language $L(R) = L(A)$ is not local. *Proof 1* – The digram aa is admissible, but it allows a valid string to contain a substring of type a^n with $n \geq 2$, whereas automaton A does not have a loop labeled only by input letters a . *Proof 2* – Automaton A is the minimal one, but it is not local, since for instance both states 1 and 2 are entered by input a . Therefore the condition, necessary for a language to be local, that the minimal automaton is local is violated.
- (e) Here is a strictly right-unilinear grammar G' , obtained directly from automaton A' , with nonterminal alphabet A, B, C (axiom A):

$$G' \left\{ \begin{array}{l} A \rightarrow a B \mid b C \\ B \rightarrow b A \\ C \rightarrow a A \mid \varepsilon \end{array} \right.$$

- (f) System of linear language equations for automaton A' , with unknown variables L_A, L_B and L_C :

$$\left\{ \begin{array}{l} L_A = a L_B \cup b L_C \\ L_B = b L_A \\ L_C = a L_A \cup \varepsilon \end{array} \right.$$

Solution of the equation system:

- Substitute equations L_B and L_C into equation L_A :

$$\begin{aligned} L_A &= a b L_A \cup b (a L_A \cup \varepsilon) \\ &= a b L_A \cup b a L_A \cup b \\ &= (a b \cup b a) L_A \cup b \\ L_B &= b L_A \\ L_C &= a L_A \cup \varepsilon \end{aligned}$$

- Solve equation L_A by Arden identity:

$$\begin{aligned} L_A &= (a b \cup b a)^* b \\ L_B &= b L_A \\ L_C &= a L_A \cup \varepsilon \end{aligned}$$

- Substitute equation L_A into equations L_B and L_C :

$$\begin{aligned} L_A &= (a b \cup b a)^* b \\ L_B &= b (a b \cup b a)^* b \\ L_C &= a (a b \cup b a)^* b \cup \varepsilon \end{aligned}$$

Since language $L_A = L(G')$, we have this regular expression R :

$$R' = (a b \mid b a)^* b$$

Other equivalent formulations of R are possible, of course.

2 Free Grammars and Pushdown Automata 20%

1. Consider the following grammar G_1 , over the two-letter terminal alphabet $\Sigma = \{ a, b \}$ and the nonterminal alphabet $V_1 = \{ A, S \}$ (axiom S):

$$G_1 \left\{ \begin{array}{l} S \rightarrow S S \mid A \\ A \rightarrow a a A b \mid a A b \mid a a b \mid a b \end{array} \right.$$

Answer the following questions:

- (a) Draw all the syntax trees of grammar G_1 for the two valid strings below:

$a a b$

$a b a b$

- (b) Say if grammar G_1 is ambiguous and suitably justify your answer. If grammar G_1 is ambiguous, write an equivalent non-ambiguous grammar.

Consider next the following grammar G_2 , over the same two-letter terminal alphabet Σ and the new nonterminal alphabet $V_2 = \{ A, B, D, E, S \}$ (axiom S):

$$G_2 \left\{ \begin{array}{l} S \rightarrow A D \\ A \rightarrow a A b \mid B \\ B \rightarrow a B b b \mid \varepsilon \\ D \rightarrow b D a \mid E \\ E \rightarrow b b E a \mid \varepsilon \end{array} \right.$$

Answer the following further questions:

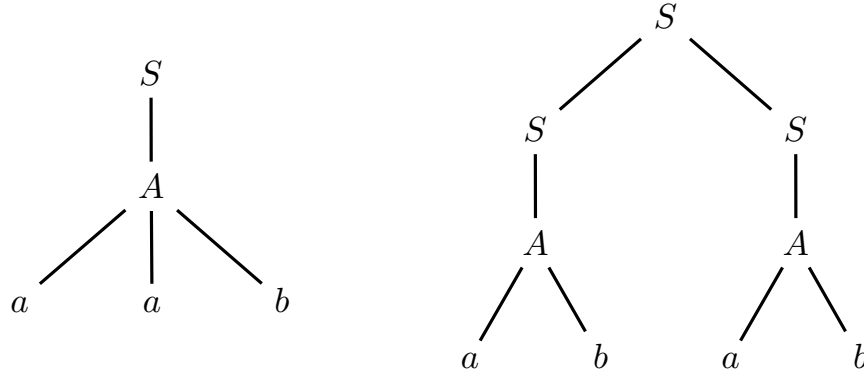
- (c) Draw all the syntax trees of grammar G_2 for the valid string below:

$a b b a$

- (d) (optional) Say if grammar G_2 is ambiguous and suitably justify your answer. If grammar G_2 is ambiguous, draw two syntax trees of the shortest possible ambiguous string of language $L(G_2)$.

Solution

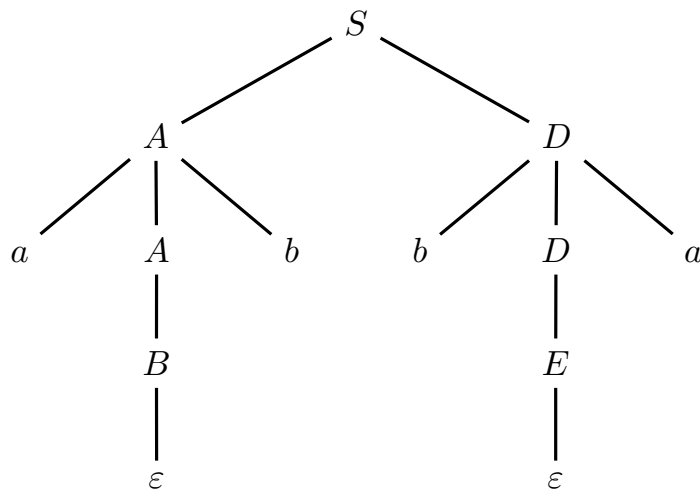
- (a) Here are the syntax trees for the strings aab and $abab$ of grammar G_1 :



- (b) Grammar G_1 is ambiguous because of the presence of the alternative $S \rightarrow S S \mid \dots$, i.e., a two-sided recursive rule. In fact, the phrase form SSS admits two distinct leftmost derivations. Moreover, it is ambiguous because of the presence of the alternatives $A \rightarrow aaAb \mid aAb \mid \dots$, which can be applied in various different orders, for instance to generate the string $aaabbb$. Here below is a non-ambiguous grammar, where the left recursion on S and the additional nonterminal B impose an order in the derivations:

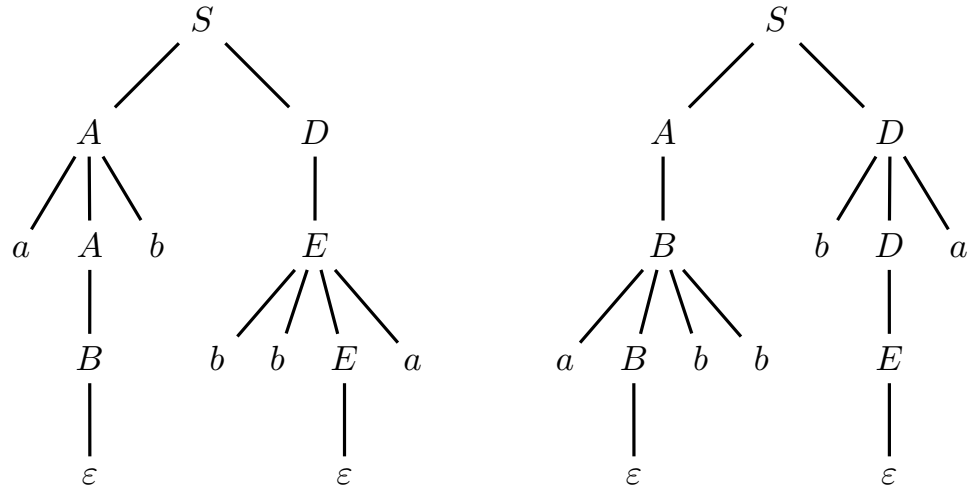
$$\left\{ \begin{array}{l} S \rightarrow AS \mid A \\ A \rightarrow aaAb \mid aab \mid B \\ B \rightarrow aBb \mid ab \end{array} \right.$$

- (c) Here is the syntax tree for the string $abba$ of grammar G_2 :



Actually, grammar G_2 is ambiguous (see the next point (d)), but the short string $abba$ is not ambiguous, thus it has only one tree. See the next point for a better understanding of the ambiguity type of G_2 .

- (d) Grammar G_2 is ambiguous, although language $L(G_2)$ is the concatenation of the two non-ambiguous languages $L(A)$ and $L(D)$ generated by the two nonterminals A and D . There are in fact cases where, if the number of letters b is greater than the number of letters a , some of the letters b in excess may be derived either from nonterminal A or from nonterminal B . As an example, consider the ambiguous string $abbb a$. Below are reported two syntax trees:



The letters b in the middle are distributed differently between the nonterminals A and D . The reader may wish to verify that the concatenation of the two languages of A and D formally satisfies the condition of concatenation ambiguity (see the textbook).

2. We aim at modelling the variant construct (also known as tagged union) exemplified below:

```
variant Tree {          # declaration of a tree variant type
    Leaf;               # empty leaf node (a void entry)
    Node {              # a node composed of various fields
        value : int,    # each field has an associated type
        left : Tree,    # the structure can be recursive
        right : Tree
    };
}
```

```
-----
# declaration of a variable T of type Tree as defined above
# and initialization of the variable T with an expression
# that matches the structure of the type
```

```
Tree T = Node (
    5,
    Node (1, Leaf, Leaf),
    Node (3, Leaf, Node (4, Leaf, Leaf))
);
```

```
-----
# match statement, which checks the actual object type of T
match T with
    | Node (x, l, r) : print (x);
    | Leaf : print ("empty tree");
```

The variant construct has the following properties:

- The variant type is a named union – each variable of type Tree can be either a Leaf (implicitly of type void) or a Node.
- Elements of all the valid types of the variant can be assigned to variables of the corresponding variant type.
- The `match with` statement allows to execute different statements for each possible type of a variant variable.
- Use the `id` and `num` terminals for the identifiers (i.e., names of functions, types, variables and fields) and the numbers, respectively.
- Assume a `STAT` nonterminal that represents the actions to be taken in the `match` statement. Nonterminal `STAT` does not need to be further expanded.
- Comments (starting with a character `#`) do not need to be included in the grammar.

Write three extended BNF grammars (EBNF), non-ambiguous, that respectively model:

- the variant type declaration
- the variant variable declaration and initialization
- and the match statement

Solution

The examples just hint the language structure and should be generalized: a variant declaration (1-st sample) may contain any number ≥ 1 of fields, and the variable declarations (2-nd sample) may be nested as recursion is allowed. Here is a straightforward solution, which uses some EBNF constructs (for instance, lists):

(a) Variant type declaration – axiom **VARIANT_T**:

```
VARIANT_T    -> 'variant' id '{' VAR_ITEM (';' VAR_ITEM)* '}'
VAR_ITEM     -> id | id '{' TYPED_VARS '}'
TYPED_VARS   -> id ':' id (';' id ':' id)*
```

A variant can contain one or more fields, each being a variable declaration.

(b) Variant variable declaration and initialization – axiom **VAR_DECL**:

```
VAR_DECL     -> id id '=' VAR_DEF ';'
VAR_DEF      -> id | id '(' PAR_DEF (';' PAR_DEF)* ')'
PAR_DEF      -> num | id | id '(' PAR_DEF (';' PAR_DEF)* ')'
```

A variable can contain one or more values, each being a scalar or a variant.

(c) Statement **match-with** – axiom **MATCH_STAT**:

```
MATCH_STAT   -> 'match' id 'with' ('|' MATCH_CLAUSE ';')+
MATCH_CLAUSE -> id ':' STAT | id '(' id (';' id)* ')' ':' STAT
```

The fields of a variant variable can be matched with one or more clauses and, in the case of correspondence, a corresponding statement is executed.

The grammars satisfy the specifications and generate the examples as particular cases.

Here is an alternative solution (of course equivalent to the former), with some rearrangement and compaction (nonterminals for the lists and optionality):

(a) Variant type declaration – axiom **VARIANT_T**:

$$\left\{ \begin{array}{l} \langle \text{VARIANT_T} \rangle \rightarrow \text{variant id } \{ \langle \text{VAR_ITEM} \rangle (; \langle \text{VAR_ITEM} \rangle)^* \} \\ \langle \text{VAR_ITEM} \rangle \rightarrow \text{id } [\langle \text{TVAR_LIST} \rangle] \\ \langle \text{TVAR_LIST} \rangle \rightarrow \{ \langle \text{TYPED_VAR} \rangle (, \langle \text{TYPED_VAR} \rangle)^* \} \\ \langle \text{TYPED_VAR} \rangle \rightarrow \text{id } : \text{id} \end{array} \right.$$

(b) Variant variable declaration and initialization – axiom **VAR_DECL**:

$$\left\{ \begin{array}{l} \langle \text{VAR_DECL} \rangle \rightarrow \text{id id } = \langle \text{VAR_DEF} \rangle ; \\ \langle \text{VAR_DEF} \rangle \rightarrow \text{id } [\langle \text{PAR_LIST} \rangle] \\ \langle \text{PAR_LIST} \rangle \rightarrow (\langle \text{PAR_DEF} \rangle (, \langle \text{PAR_DEF} \rangle)^*) \\ \langle \text{PAR_DEF} \rangle \rightarrow \text{num } | \langle \text{VAR_DEF} \rangle \end{array} \right.$$

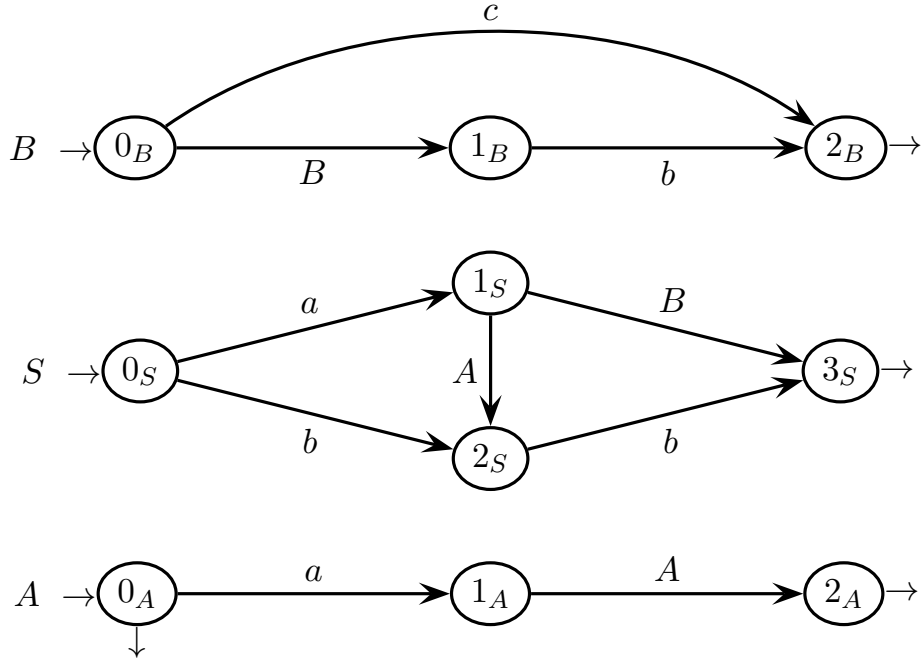
(c) Statement **match-with** – axiom **MATCH_STAT**:

$$\left\{ \begin{array}{l} \langle \text{MATCH_STAT} \rangle \rightarrow \text{match id with } (| \langle \text{MATCH_CLAUSE} \rangle ;)^+ \\ \langle \text{MATCH_CLAUSE} \rangle \rightarrow \text{id } [((\text{id } (, \text{id })^*)) : \langle \text{STAT} \rangle \end{array} \right.$$

where the square brackets $[]$ indicate optionality.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following EBNF grammar G , presented as a machine net over the three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ and the nonterminal alphabet $V = \{ A, B, S \}$ (axiom S):



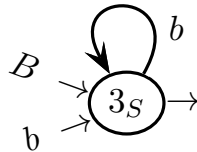
Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) List all the short valid strings of length ≤ 3 that are generated by grammar G .
- (b) Prove that grammar G is of type ELR (1), by designing its complete pilot (three pilot m-states are already provided, complete the rest of the pilot). Are there multiple transitions ? Are there convergent transitions ? Explain your answers.
- (c) Find all the guide sets on the call arcs and exit arrows of the net, for $k = 1$. Determine if grammar G is of type ELL (k) for some $k \geq 1$. Explain your answer.
- (d) Execute the Earley analysis for the valid string of grammar G below (compute the complete Earley vector, with all the items):

$a a b$

Show the reconstructed syntax tree of the string.

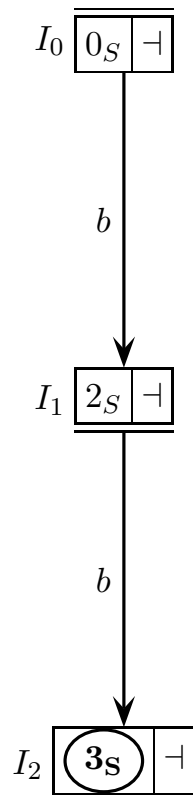
- (e) (optional) Suppose to add to the machine M_S of grammar G a self-loop on state 3_S , labeled with terminal b :



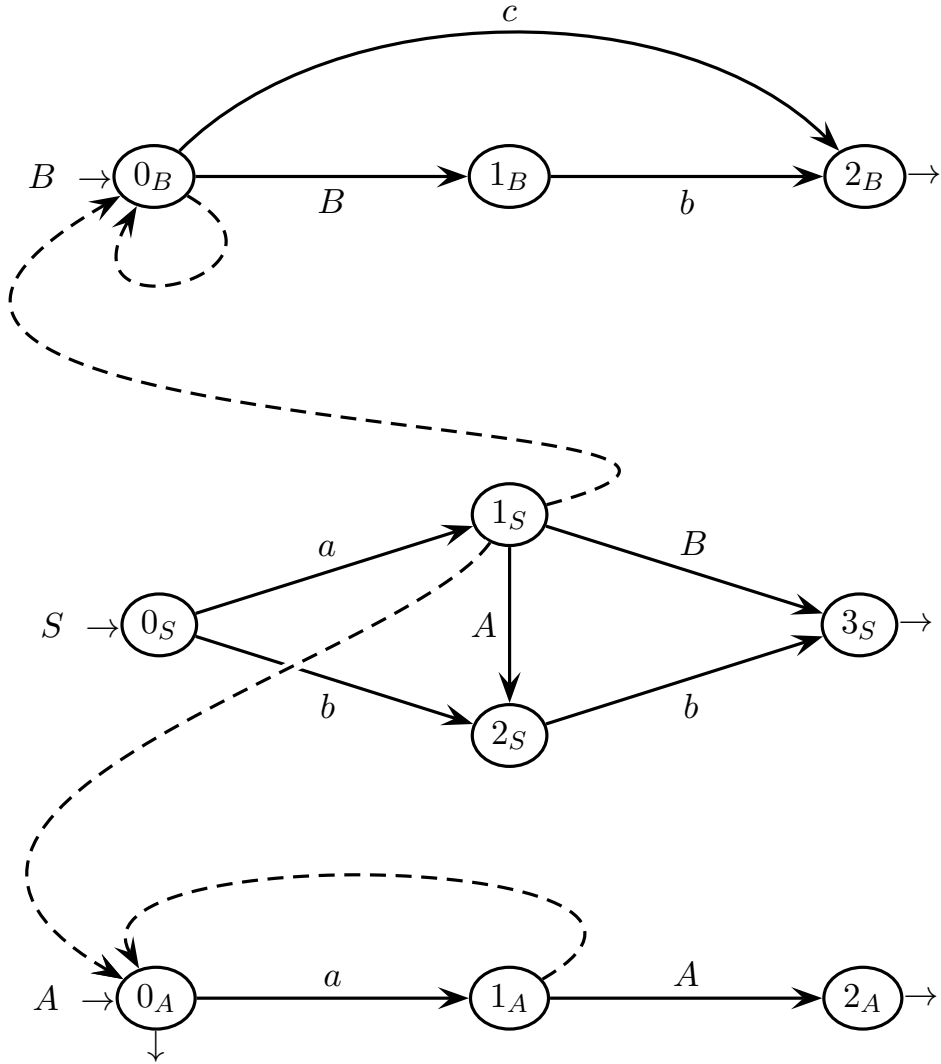
Is the so-obtained new grammar G' deterministic ? Does it have any ELR (1) conflicts ? Explain your answers.

short strings of length ≤ 3 – question (a)

pilot to be completed – question (b)



guide sets to be computed – question (c)



space for answering question (d)

complete Earley vector of aab (to be filled in – number of rows not significant)

0	a	1	a	2	b	3

space for answering question (e)

free space for continuation or correction if necessary – any questions

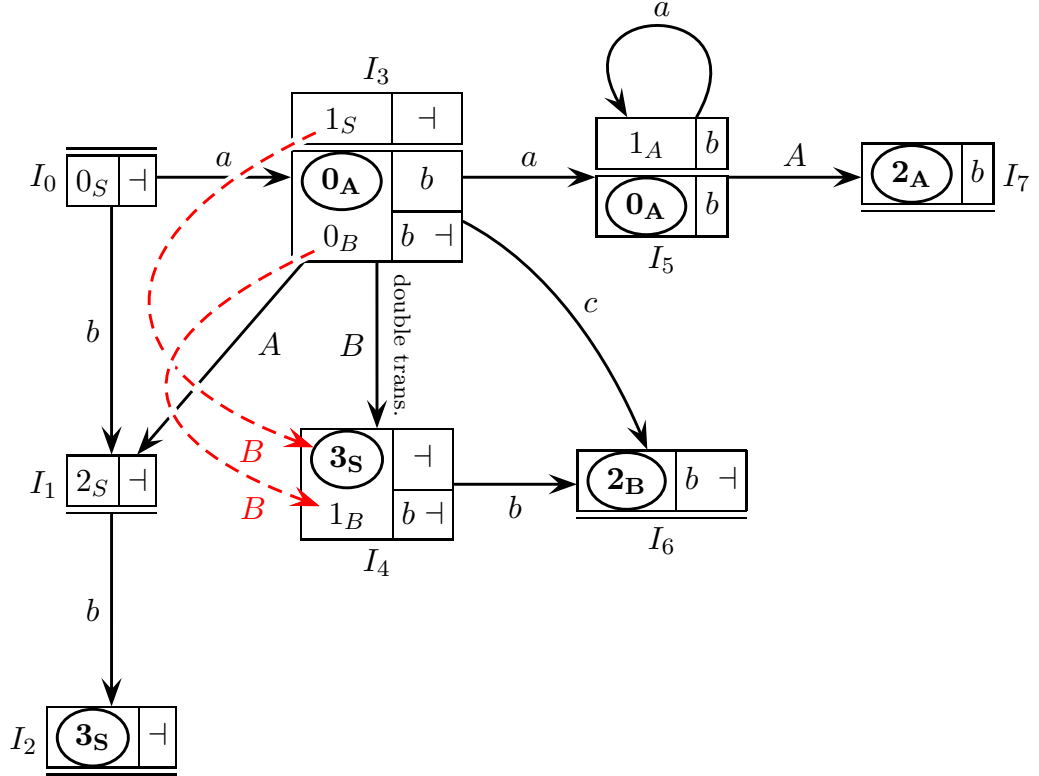
Solution

- (a) Here are the five short valid strings (of length ≤ 3): ab , ac , bb , aab and acb . Their correctness is rather evident, however here is a derivation for the last one:

$$S \xRightarrow{S \rightarrow aB} aB \xRightarrow{B \rightarrow Bb} aBb \xRightarrow{B \rightarrow c} acb$$

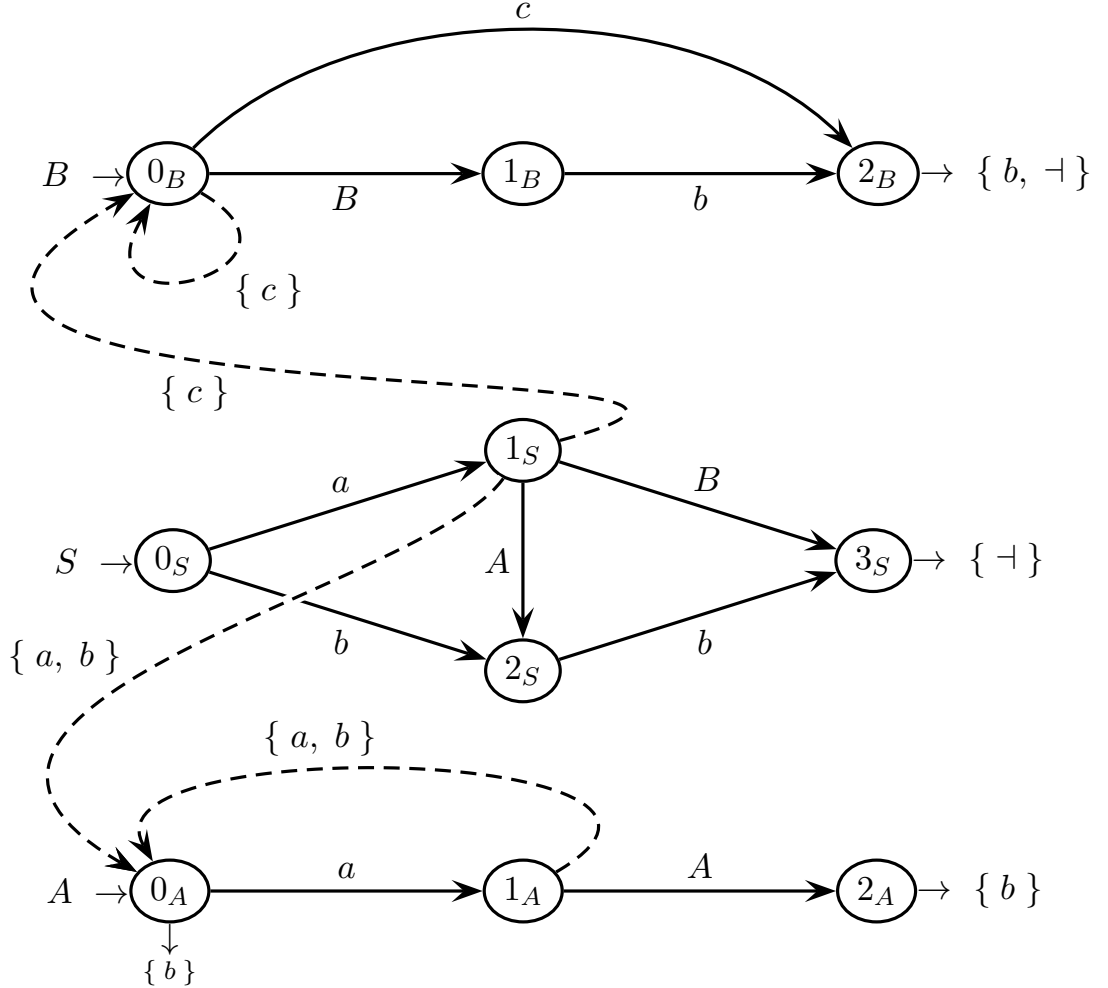
where the three rules correspond to as many accepting paths in the machines M_S , M_B and (again) M_B , respectively. Similarly, the other strings have a derivation.

- (b) Here is the complete pilot of grammar G , with eight m-states I_0 – I_7 :



The pilot does not have any conflicts (SR, RR or Convergence), therefore grammar G is of type ELR(1). The pilot transition $I_3 \xrightarrow{B} I_4$ is double, as it is composed of the two machine transitions $1_S \xrightarrow{B} 3_S$ and $0_B \xrightarrow{B} 1_B$ (see the dashed red arrows), but it is not convergent as the two destination items 3_S and 1_B in the m-state I_4 are different. Since all the remaining m-states of the pilot have a base containing only one item, there are not any other multiple transitions. As anyway not all transitions are single, the single transition property (STP) fails.

(c) Here are the guide sets for look-ahead depth $k = 1$:



The machine net has a conflict on input c between the two arcs from state 0_B , i.e., the terminal shift arc $0_B \xrightarrow{c} 2_B$ and the call arc $0_B \xrightarrow{\{c\}} 0_B$ (associated to the nonterminal shift arc $0_B \xrightarrow{B} 1_B$), therefore grammar G is not of type ELL(1). One could also notice that machine M_B exhibits an (immediate) left recursion, as the rule corresponding to machine M_B is $B \rightarrow Bb \mid c$, the first alternative being left-recursive, which always prevents a grammar from being ELL(1). Grammar G is not of type ELL(2) either. In fact, the string cb of length 2 belongs to the guide sets of the two arcs from state 0_B , for these reasons:

- the c -arc enters state 2_B , and a letter b occurs in the exit guide set of 2_B
- the B -arc enters state 1_B , and a letter b labels the shift arc $1_B \rightarrow 2_B$

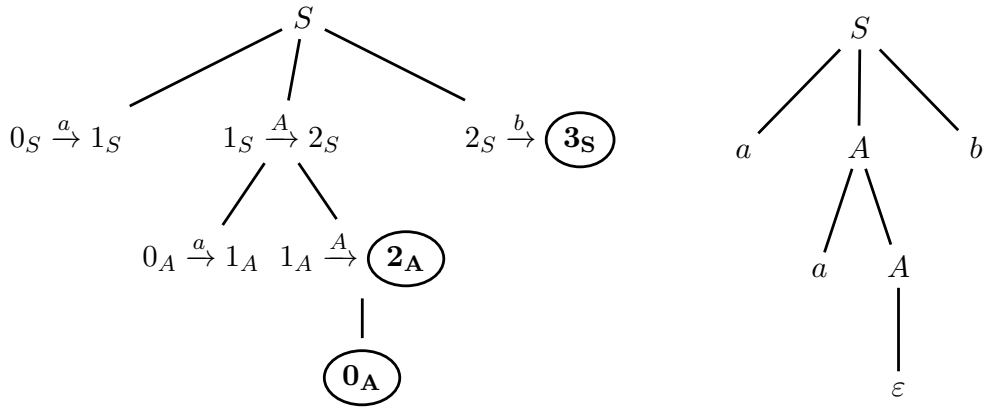
Thus for both such arcs an input letter b is expected after the input letter c .

It turns out eventually, that grammar G is not of type ELL(k) for all $k \geq 1$. In fact, the string cb^{k-1} (with $k \geq 1$) with any number $k-1 \geq 0$ of letters b belongs to the guide sets of the two arcs from state 0_B ; for $k = 1, 2$ the cases ELL(1), ELL(2) are reobtained, respectively. The general reason is that for both such arcs all the letters b after the first one are recursively generated by machine M_B , which corresponds to the left-recursive rule $B \rightarrow Bb \mid c$ deriving as $B \xRightarrow{+} cb^*$.

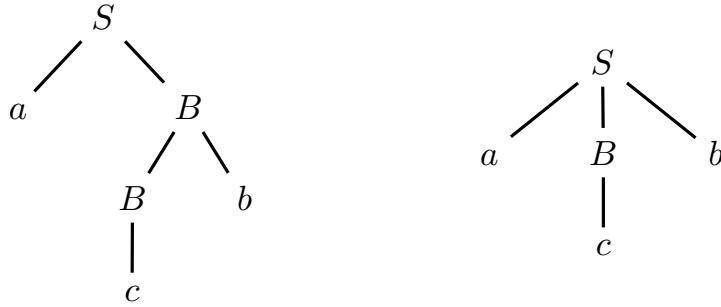
(d) Here is the Earley vector of string $a a b$ (complete, including the useless items):

0	a	1	a	2	b	3																					
<table><tr><td>0_S</td><td>0</td></tr><tr><td colspan="2" style="background-color: #cccccc;"></td></tr></table>	0_S	0			<table><tr><td>1_S</td><td>0</td></tr><tr><td>0_A</td><td>1</td></tr><tr><td>0_B</td><td>1</td></tr><tr><td>2_S</td><td>0</td></tr></table>	1_S	0	0_A	1	0_B	1	2_S	0	<table><tr><td>1_A</td><td>1</td></tr><tr><td>0_A</td><td>2</td></tr><tr><td>2_A</td><td>1</td></tr><tr><td>2_S</td><td>0</td></tr></table>	1_A	1	0_A	2	2_A	1	2_S	0	<table><tr><td>3_S</td><td>0</td></tr><tr><td colspan="2" style="background-color: #cccccc;"></td></tr></table>	3_S	0		
0_S	0																										
1_S	0																										
0_A	1																										
0_B	1																										
2_S	0																										
1_A	1																										
0_A	2																										
2_A	1																										
2_S	0																										
3_S	0																										

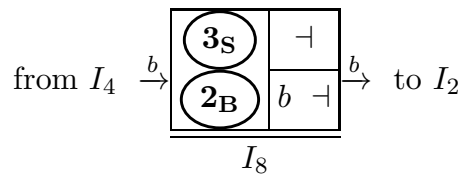
The string is accepted because the final axiomatic state 3_S with pointer 0 occurs in the last vector state. Notice there are null reductions. Reconstructed tree:



(e) Adding the self-loop makes grammar G' ambiguous. For instance, string $a c b$, which is generated by the original grammar G , but unambiguously (see point (a)), is also generated by the new grammar G' , but for G' this string becomes ambiguous. Therefore grammar G' is nondeterministic. To prove, draw the two trees of string $a c b$ (one uses recursion on B , the other does not) for G' . Trees:



Of course, we expect to find at least one conflict in the pilot of grammar G' . The new pilot is similar to that of G , but now m-state I_2 has a (non-conflicting) self-loop labeled by terminal b , while a new m-state I_8 appears:



The m-state I_8 has two conflicts: shift-reduce on terminal b and reduce-reduce on the terminator $-|$, which are due to the grammar ambiguity.

4 Language Translation and Semantic Analysis 20%

1. A *bracket expression* is an odd-length list of balanced bracket nests. A nest can be:
 - empty: it consists of a pair of round brackets with nothing inside it, or
 - non-empty: a pair of round brackets that *recursively* encloses a bracket expression

For example, the following sample bracket expression (left):

$$\underbrace{\underbrace{() \quad \underbrace{(\quad \overbrace{((\quad))}^{\text{three sub-nests}} \quad (\quad))}_{\text{2-nd}} \quad ()}_{\text{3-rd}}}_{\text{1-st}} \quad \underbrace{\hspace{10em}}_{\text{three nests}} \quad G^s \quad \begin{cases} S \rightarrow N S N & N \rightarrow (S) \\ S \rightarrow (S) & N \rightarrow () \\ S \rightarrow () \end{cases}$$

is a list of three bracket nests. The 1-st nest is empty, the 2-nd is non-empty and includes a list of three bracket sub-nests (the first of which is non-empty, whereas the second and third are empty), and the 3-rd is non-empty, as it includes an empty nest. The source grammar G^s above (right) generates bracket expressions (axiom S).

Notice that, as the number of (top level) bracket nests in a bracket expression is odd, exactly one of them is central. Thus there exist exactly four types of bracket nests: central empty, central non-empty, non-central empty and non-central non-empty. Therefore the above sample bracket expression includes:

- four central bracket nests, of which three are empty and one is non-empty
- four non-central bracket nests, of which two are empty and two are non-empty

We want to define a translation τ where the top bracket pairs of all bracket nests are translated as follows, depending on the four types of bracket nests:

	central	non-central
empty	$[]$	$[\]$
non-empty	$\lceil \rceil$	$\{ \}$

Therefore the above sample bracket expression is translated as follows:

$$[] \left[\left\{ \lceil \rceil \right\} \lceil \rceil [] \right] \left\{ \lceil \rceil \right\}$$

Answer the following questions:

- (a) Design a target grammar G^t , matching G^s , that defines translation τ .
- (b) Draw the source syntax tree for the sample string below:

$$() \ (()) \ (())$$

and the corresponding target syntax tree.

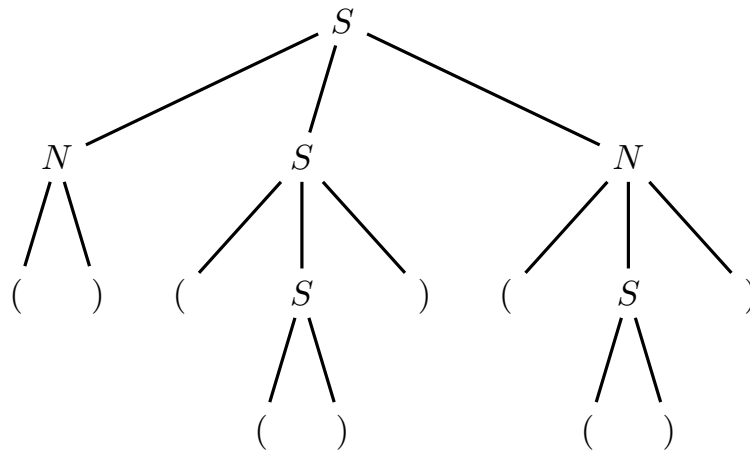
- (c) (optional) Say if translation τ can be computed by a top-down (i.e., recursive-descent or ELL) parser enriched with write actions. Suitably justify your answer.

Solution

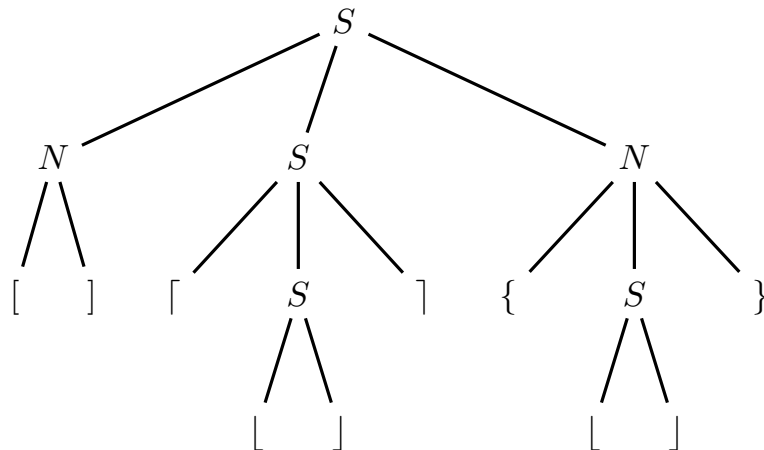
- (a) Here is the target grammar G^t (on the right) that matches the source grammar G^s (on the left):

$$G^s \left\{ \begin{array}{l} S \rightarrow N S N \\ S \rightarrow (S) \\ S \rightarrow () \\ N \rightarrow (S) \\ N \rightarrow () \end{array} \right. \quad G^t \left\{ \begin{array}{ll} S \rightarrow N S N & \\ S \rightarrow [S] & // \text{ central non-empty} \\ S \rightarrow [] & // \text{ central empty} \\ N \rightarrow \{ S \} & // \text{ non-central non-empty} \\ N \rightarrow [] & // \text{ non-central empty} \end{array} \right.$$

- (b) Here is the source syntax tree for the sample string:



and the corresponding target syntax tree:



- (c) Translation τ cannot be computed by a top-down parser enriched with write actions, because the source grammar is not $\text{ELL}(k)$ for any k . In fact, the alternatives $S \rightarrow N S N$ and $S \rightarrow (S)$ share the string $(^k$ for any $k \geq 1$, and the situation does not change even if the grammar is rewritten as a machine net.

2. Consider the following grammar of integer arithmetic expressions, with addition “+”, multiplication “ $\times$ ” and subexpressions enclosed in round brackets, where the variables and constants (positive or zero) are schematized by terminal `num` (axiom E):

$$\left\{ \begin{array}{l} 1: E \rightarrow T + E \\ 2: E \rightarrow T \\ 3: T \rightarrow F \times T \\ 4: T \rightarrow F \\ 5: F \rightarrow (E) \\ 6: F \rightarrow \text{num} \end{array} \right.$$

We want to define an attribute grammar that computes the *minimum bit width* of each intermediate integer (positive or zero) value, i.e., the smallest number of bits necessary for representing that value without error. The minimum bit width is described as an integer attribute w , of type *left* (synthesized), associated to the nonterminals E , T and F . The following specifications must be taken into account:

- For each terminal `num`, if n is the decimal value associated with the token `num`, the minimum bit width is defined as the smallest integer $b \geq 1$ such that $n < 2^b$ (n strictly less than 2^b). For instance, for $n = 3_{dec} = 11_{bin}$ we have $b = 2$ bits, and for $n = 4_{dec} = 100_{bin}$, we have $b = 3$ bits.
- The sum of two integer numbers can be expressed on at most as many bits as the larger number has, plus one bit.
- The product of two integer numbers can be expressed on at most twice as many bits as the larger number has.

Answer the following questions (use the tables / trees / spaces on the next pages):

- Define the semantic rules for computing the attribute w in each grammar rule.
- Decorate the given syntax tree with the values of the attribute w (in all the tree nodes) for the following arithmetic expression:

$$11 \times 3 + 13 \times (4 + 9)$$

- (optional) Is the attribute grammar one-sweep ? Is it possible to write a recursive descent integrated syntactic-semantic analyzer for the grammar ? If so, explain why and write the integrated analyzer (or some significant part thereof).

semantic functions - question (a)

syntax *semantic rules for the attribute*

$$1: \quad E_0 \rightarrow T_1 + E_2$$

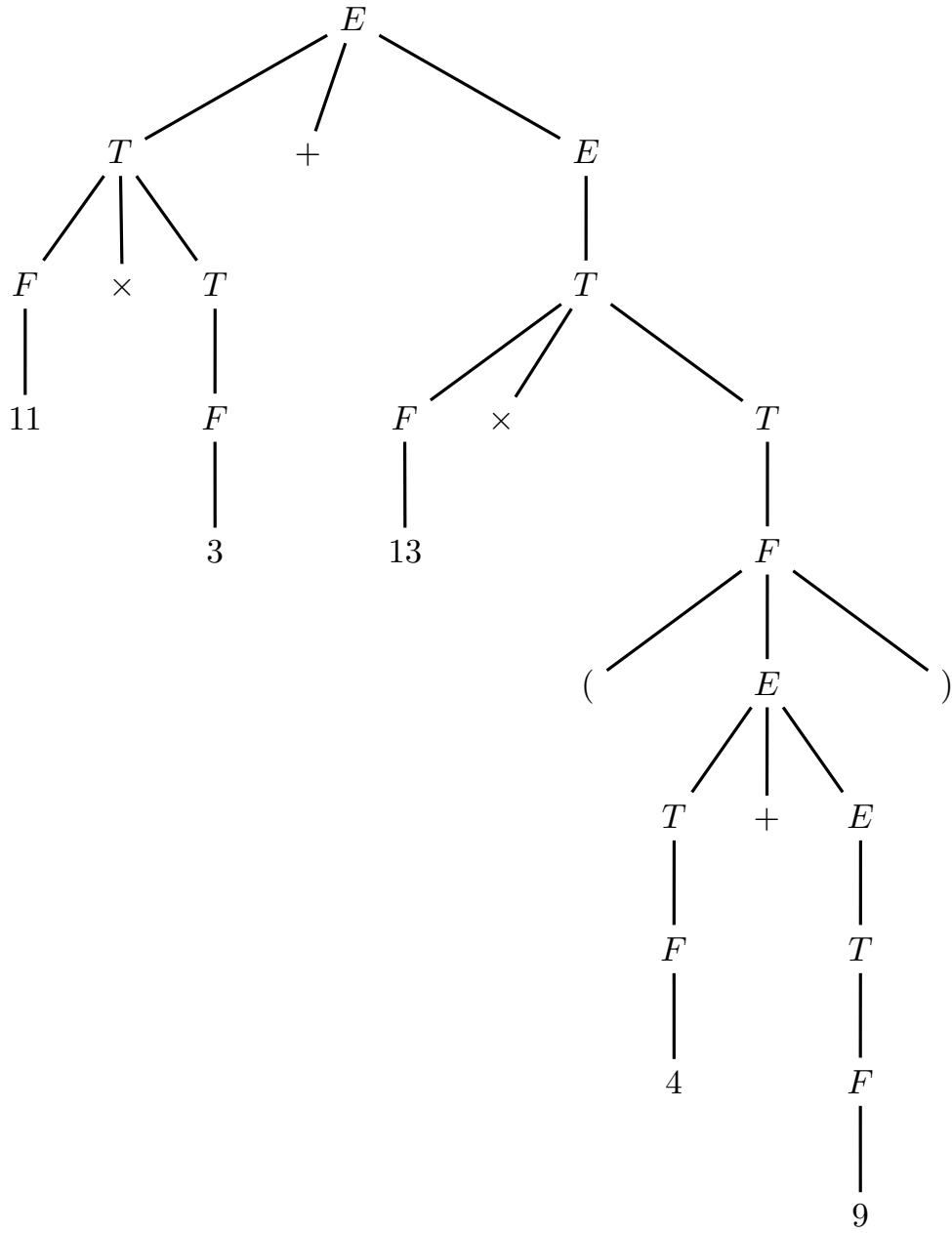
$$2: \quad E_0 \rightarrow T_1$$

$$3: \quad T_0 \rightarrow F_1 \times T_2$$

$$4: \quad T_0 \rightarrow F_1$$

$$5: \quad F_0 \rightarrow (E_1)$$

tree to be decorated - question (b)



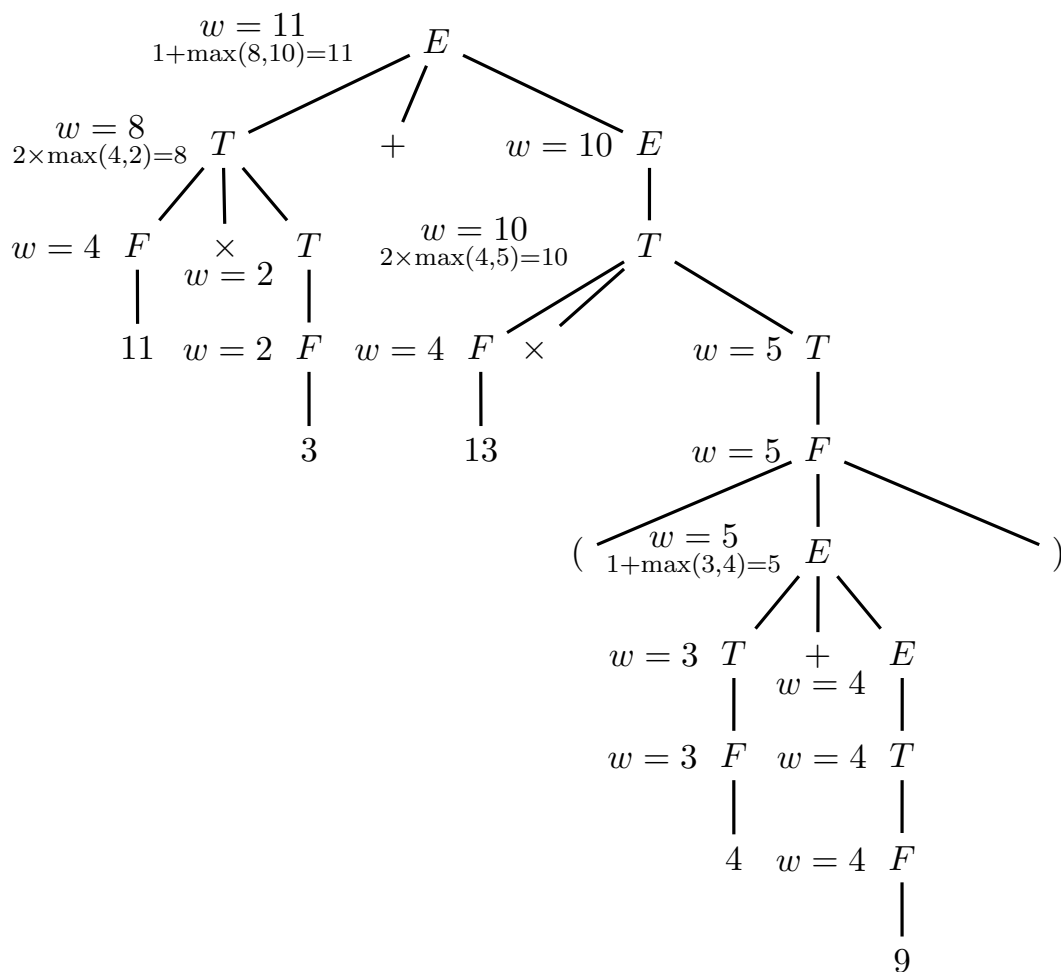
space for answering question (c)

Solution

- (a) Here are the semantic functions for attribute w (min bit weight), of type left:

#	<i>syntax</i>	<i>semantic rules for the attribute w</i>
1:	$E_0 \rightarrow T_1 + E_2$	$w_0 := 1 + \max(w_1, w_2)$
2:	$E_0 \rightarrow T_1$	$w_0 := w_1$
3:	$T_0 \rightarrow F_1 \times T_2$	$w_0 := 2 \times \max(w_1, w_2)$
4:	$T_0 \rightarrow F_1$	$w_0 := w_1$
5:	$F_0 \rightarrow (E_1)$	$w_0 := w_1$
6:	$F_0 \rightarrow \text{num}$	$w_0 := \text{the smallest integer } b \geq 1 \text{ such that } n < 2^b$

- (b) Remember that in the binary representation of natural numbers we have $11_{\text{dec}} = 1011_{\text{bin}}$, $3_{\text{dec}} = 11_{\text{bin}}$, $13_{\text{dec}} = 1101_{\text{bin}}$, $4_{\text{dec}} = 100_{\text{bin}}$ and $9_{\text{dec}} = 1001_{\text{bin}}$, i.e., those values need 4, 2, 4, 3 and 4 bits, respectively. Then the decorated tree is:



Therefore the result of the expressions requires no more that 11 bits.

- (c) The attribute grammar is purely synthesized, hence it is trivially one-sweep as there are no inherited (right) attributes. Here is the one-sweep semantic analyzer, consisting of the semantic procedures for nonterminals E , T and F . The synthesized (left) attribute w is an output parameter (no input parameters):

```

procedure  $E$  (out  $w_0$ : integer)
var  $w_1, w_2$ : integer
switch rule number do
  case 1 do      //  $E \rightarrow T + E$ 
    call  $T(w_1)$ 
    call  $E(w_2)$ 
     $w_0 := 1 + \max(w_1, w_2)$ 
  case 2 do      //  $E \rightarrow T$ 
    call  $T(w_1)$ 
     $w_0 := w_1$ 
  else error
end procedure  $E$ 

procedure  $T$  (out  $w_0$ : integer)
var  $w_1, w_2$ : integer
switch rule number do
  case 3 do      //  $T \rightarrow F \times T$ 
    call  $F(w_1)$ 
    call  $T(w_2)$ 
     $w_0 := 2 \times \max(w_1, w_2)$ 
  case 4 do      //  $T \rightarrow F$ 
    call  $F(w_1)$ 
     $w_0 := w_1$ 
  else error
end procedure  $T$ 

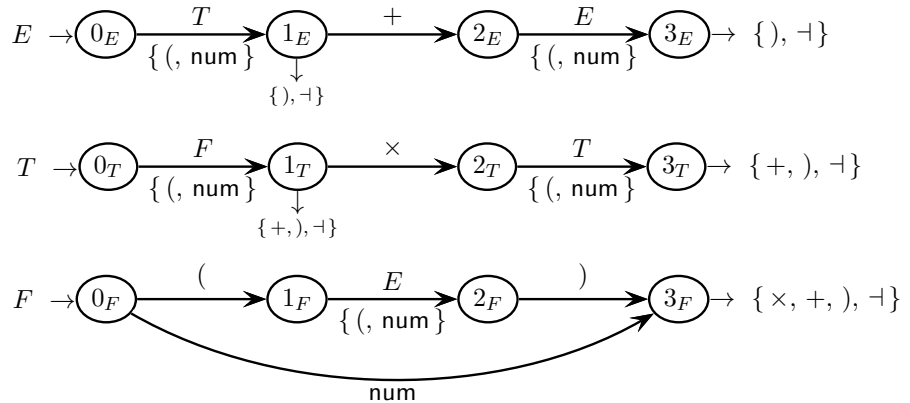
```

```

procedure  $F$  (out  $w_0$ : integer)
var  $w_1$ : integer
switch rule number do
  case 5 do      //  $F \rightarrow (E)$ 
    call  $E(w_1)$ 
     $w_0 := w_1$ 
  case 6 do      //  $F \rightarrow \text{num}$ 
     $w_0 := \text{the smallest integer } b \geq 1 \text{ such that } n < 2^b$ 
  else error
end procedure  $F$ 

```

Concerning the syntactic analysis and its integration with semantic analysis, we can immediately notice that the grammar is not of type LL(1), as rules 1 and 2 have overlapping guide sets (similarly for rules 3 and 4). However, the grammar may be of type ELL. Here is the corresponding machine net with guide sets (the call arcs are not shown, so their guide sets are written below the corresponding nonterminal shift arcs) and the evaluation of the ELL condition:



The machine net does not have any guide set conflicts, thus the grammar is of type ELL (1). Therefore a recursive-descent syntactic analyzer can be designed for the grammar (see below).

Again, since the grammar is purely synthesized, the L condition is trivially satisfied (as there are no inherited attributes). Since the grammar is both ELL and L, it is possible to integrate recursive-descent syntax analysis with one-sweep semantic analysis. Here are the recursive-descent syntactic analyzer (left) and the integration with the one-sweep semantic analyzer (right), widely commented:

<pre> // syntactic procedure procedure E // state 0_E rules 1 and 2 if cc ∈ { (, num } then call T // state 1_E rules 1 and 2 if cc ∈ { + } then cc := next // state 2_E rule 1 if cc ∈ { (, num } then call E // state 3_E rule 1 if cc ∈ {), ÷ } then return else error else error // state 1_E rule 2 else if cc ∈ {), ÷ } then return else error else error end procedure E </pre>	<pre> // integrated procedure procedure E (out w₀: integer) var w₁, w₂: integer if (cc ∈ { (, num }) then call T(w₁) if cc ∈ { + } then cc := next if cc ∈ { (, num } then call E(w₂) if cc ∈ {), ÷ } then w₀ := 1 + max(w₁, w₂) return else error else error else if cc ∈ {), ÷ } then w₀ := w₁ return else error else error end procedure E </pre>
--	---

The procedure could be optimized, of course. For instance all the error cases could be grouped at the end, since each of them is alternative to a return:

<pre> procedure E (out w₀: integer) var w₁, w₂: integer if cc ∈ { (, num } then call T(w₁) if cc ∈ { + } then cc := next if cc ∈ { (, num } then call E(w₂) if cc ∈ {), ÷ } then w₀ := 1 + max(w₁, w₂) return else if cc ∈ {), ÷ } then w₀ := w₁ return error end procedure E </pre>	<pre> // integrated procedure </pre>
--	--------------------------------------

Since there are only synthesized attributes, i.e., output parameters, the corresponding semantic functions are computed immediately before each return point.

The recursive-descent syntactic procedures for the other two nonterminals T and F , and their integration with the corresponding semantic procedures, can be easily designed as well. We leave the procedures for T to the reader, as this case is structurally similar to that of E , though with different guide sets (which are visible on the machine net). Here are the procedures for F instead:

<pre> // syntactic procedure procedure F // state 0_F rules 5 and 6 if $cc \in \{ (\}$ then $cc := next$ // state 1_F rule 5 if $cc \in \{ (, num \}$ then $call\ E$ // state 2_F rule 5 if $cc \in \{) \}$ then $cc := next$ else error else error else if $cc \in \{ num \}$ then $cc := next$ else error // state 3_F rules 5 and 6 if $cc \in \{ \times, +,), \neg \}$ then return else error end procedure F </pre>	<pre> // integrated procedure procedure F (out w_0: integer) var w_1: integer if $cc \in \{ (\}$ then $cc := next$ if $cc \in \{ (, num \}$ then $call\ E(w_1)$ if $cc \in \{) \}$ then $cc := next$ $w_0 := w_1$ else error else error else if $cc \in \{ num \}$ then $cc := next$ $w_0 := \text{the smallest } \dots$ else error if $cc \in \{ \times, +,), \neg \}$ then return else error end procedure F </pre>
--	---

These procedures could be optimized as well, of course. For instance, the local variable w_1 and the subsequent assignment “ $w_0 := w_1$ ” can be eliminated, just equivalently writing “ $call\ E(w_0)$ ”. Here is such a simplification:

<pre> procedure F (out w_0: integer) if $cc \in \{ (\}$ then $cc := next$ if $cc \in \{ (, num \}$ then $call\ E(w_0)$ if $cc \in \{) \}$ then $cc := next$ else error else error else if $cc \in \{ num \}$ then $cc := next$ $w_0 := \text{the smallest integer } b \geq 1 \text{ such that } n < 2^b$ else error if $cc \in \{ \times, +,), \neg \}$ then return else error end procedure F </pre>	<pre> // integrated procedure // simplification </pre>
---	--

which could also have been applied to the pure semantic procedure shown before.